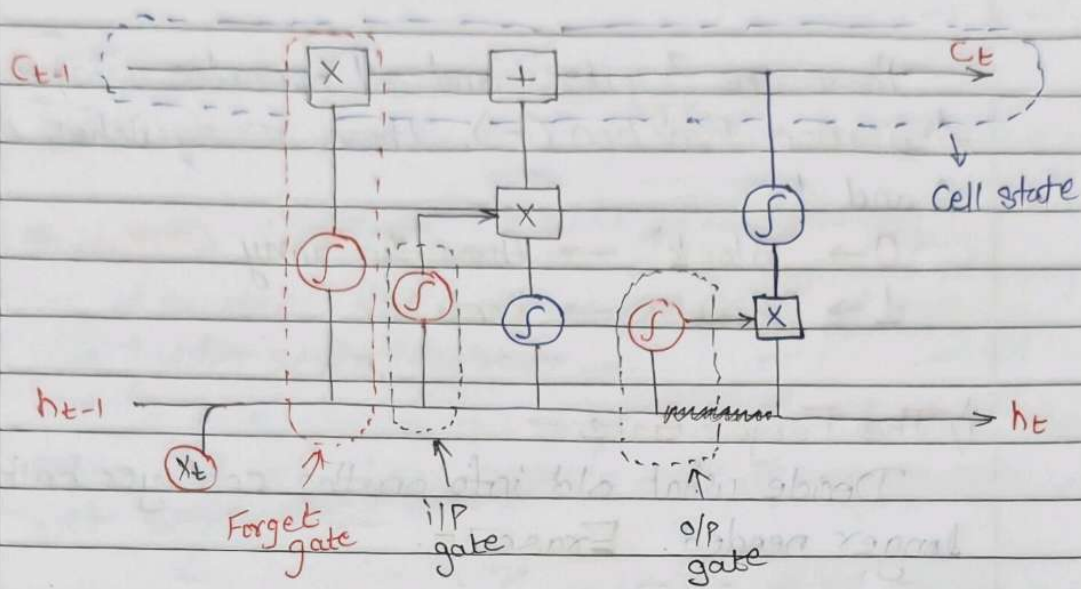


Date: / /



$\sigma \rightarrow$ sigmoid

$\tanh \rightarrow$ tanh

$\times \rightarrow$ Pointwise multiplication

$+$ $\rightarrow$ Pointwise addition

Transformers

While LSTMs solved the problem of the vanishing gradient, they still are sequential.

They had to process data one word at a time and cannot be parallelized across GPUs (making them slow).

In 2017, researchers at Google published a landmark paper titled "**Attention is All you Need**". They proposed throwing away the recurrent loop entirely and ~~proper~~ processing the whole sequence at once.

How transformers work?

* How transformers work?

Imagine translating a book.

- An RNN/LSTM reads the book exactly like a human:
word 1 $\Rightarrow$ word 2 $\Rightarrow$ word 3
- Transformer looks at the entire page simultaneously. ^{?)}

Because Xmers doesn't have to wait for a word to finish, it utilizes **Parallel Processing**.

This allows Xmers to be trained across 1000s of GPUs at lightning speed, making incredibly large models possible.

If it reads everything at once, how does it understand the order of the words or the context?

- It's because of the engine of a Xmer called **Self-Attention**.

Self-attention allows every single word in a sequence to look at every other word to figure out its true meaning. It calculates a mathematical "attention score" to decide which words are most relevant to each other.

Example: Context is Everything

1. I sat by the bank of the river.
2. I deposited money in the bank.

During processing, the word "bank" reaches out to every other word. It calculates a high attention score for "river", realizing it is a body of water, not a financial institution.

Date: / /

	Attention score	Context Word
Bank	10%	The
(Targeted Word)	80%	river
	5%	of
	5%	the

The word 'bank' absorbs 80% of its context from "river", perfectly clarifying its meaning.

* Architecture of Xmers.

To make this parallel, attention-based system work, a std. Xmer block relies on 3 advanced mechanisms!

1. Positional Encoding (The timestamps)

- Because Xmers ingests all the words at the exact same time, it would naturally scramble the sentence into a random soup of words.

- To prevent this, before the words enter the nlu, the Xmer stamps each word with a mathematical "Position ID" (often using sine and cosine wave functions).

- This allows the nlu to process everything simultaneously while still remembering the exact order of the sequence.

2. Multi-head attention (and the Q, K, V Math)

- To calculate those attention scores, the Xmer uses 3 specific vectors for every word: a Query (Q), a Key (K), and a value (V).

i) Query (Q): What ^{a word} I am looking for (e.g., "I am a verb looking for my noun").

ii) Key (K): What I am offering/holding (e.g., "I am a noun")

iii) Value (V): What I actually mean.

* Mathematical formula:

Self-attention calculates the dot product of the Queries and keys to find the match, scales it down, and multiplies it by the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

How the math works?

The formula executes a 4-step process to blend the meaning of a sentence together.

Step 1: The Match (QK^T)

The n/w multiplies the Queries (Q) by the keys (K). The T just stands for 'Transposed'.

• Working → It calculates a 'raw compatibility' score b/w every word. If the query for 'bank' mathematically aligns with the key for 'river', they get a massive score.

Step 2: The stabilizer ($1/\sqrt{d_k}$)

Next, we divide that raw score by the square root of the dimension size ($\sqrt{d_k}$).

• Working → When we multiply large matrices in step 1, the nos. can explode into the massive extremes. This step scales down the nos so the n/w remains stable during training.

Corrected at next page

Incorrect (Step 2 prevents the problem of ~~the exploding~~ gradient. $\rightarrow$ opp. of vanishing gradient.) **X**

Step 3: The Filter (softmax)

We push those scores through a softmax function.

• Working: It squishes all the raw scores into clean percentages (blw 0 and 1) that adds up to 100%.
Now the word "bank" might assign 80% of its attention to "river", 15% to "sat", and 5% to "the".

here the
c/p looks like
0.8, 0.15, 0.05

Step 4: The Final blend ($\times V$)

Finally, we multiply those clean %ages by the values.

• Working: It takes the actual meaning of the words and dials their volume up and down based on the percentages.

Takes the meaning
of river/word and
multiplies by
say, 0.8

"River" gets its volume turned up to 80%, while "the" is muted to 5%. All these adjusted meanings are added together to create a brand new, context-rich understanding of the word "bank".

$$\text{So, Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The n/w runs multiple "heads" of this attention equation ^{at} the same time.

• Head 1 $\rightarrow$ It might pay attention to grammatical structures (linking verb to nouns).

• Head 2 $\rightarrow$ Might pay attention to emotional tone.

• Head 3 $\rightarrow$ Might pay attention to context.

✓ Ahead / 3 /

By analyzing multiple perspective at once, it builds an incredibly rich understanding of data.

3. The Feed-Forward n/w:

After the self-attention mechanism figures out how all the words are related to each other, the data is passed through a standard Feed-Forward NN.

This acts as a processing layer that refines the new, context-heavy understanding of the words before passing it to the next block.

—x—

Correction : →

Step 2 prevents the problem of vanishing gradient inside the softmax funcⁿ.

If the raw scores from QK^T get too large (ex. 10, 20, 30), the softmax gets overwhelmed and pushes the outputs to extremes 1s and 0s (becomes 0, 0, 1).

The model becomes 100% confident (though incorrect) and hence, decides not to change anything. (gradient drops to zero)

This flattens the mathematical slope to zero, causing the n/w to freeze and completely stop learning.

Dividing by $\sqrt{d_k}$ shrinks the numbers so the slope stays active.

÷ by $\sqrt{d_k} \Rightarrow 10, 20, 30$ becomes say 1, 2, 3.

softmax $\Rightarrow 0.9, 0.24, 0.67$